Ulsan-Fault earthquake catalog pipeline

Reference manual: detection → association → location → relocation (HypoDD dt.ct + dt.cc)

Long-term (2010–present) catalog for the Ulsan Fault, SE Korea — rev. 2026-08-04 · `src/ufpipe`

`ufpipe` (`src/ufpipe`) is the **end-to-end pipeline**: six stages, detection → association → augment → PHS → locate → **relocate** (HypoDD dt.ct + dt.cc). The final `relocate` stage produces the double-difference relocated catalog. The heavy dt.cc/xcorr/HypoDD *engine* is external (`15.PocketQuake` korea-cluster-relocation, HypoDD v2.1beta); ufpipe orchestrates it. The relocate stage is **self-fed**: it builds its inputs from ufpipe's OWN detection+association (`ufpipe.reloc_inputs`) and runs the orchestration in `src/ufpipe/reloc_driver/`; working dirs + results go to `outputs/reloc/`. `detection_test/` is a **fully frozen pilot archive** (the 2016 4-picker comparison; `lib/` DEPRECATED, see `detection_test/lib/DEPRECATED.md`). The relocation internals have a dedicated guide: `src/ufpipe/reloc_driver/PIPELINE.md` + the pilot's `study_guide.pdf`.

Data scope (verified 2026-07): the waveform archive covers **2010–2024**, ending at 2024 day 305 (31 Oct 2024). No 2025+ data is present in any network directory, so 2024's 305 daily pick files are complete for the data that exists.

Contents

1 What the pipeline does

Build an **internally consistent, long-term earthquake catalog** for the Ulsan Fault region (SE Korea), 2010–present, from continuous seismic waveforms, using AI phase pickers so small events are detected uniformly even as the recording network grows, and finally **relocate** the catalog with double-difference (HypoDD dt.ct + dt.cc) for sub-100 m relative precision. Every stage is scripted and parameterized; results reproducible.

Detection	**Association**	**Location**	**QC**	**Relocation**
PhaseNet / PhaseNet+	PyOcto (+augment)	HYPOINVERSE	`uf_cluster`	HypoDD dt.ct + dt.cc
picks →	*events+assign* →	*.sum/.arc* →	*well-located* →	*hypoDD.reloc*

1.1 `ufpipe` vs `detection_test`

`ufpipe` is the production end-to-end pipeline; everything live is under `src/`, including the relocation orchestrator `src/ufpipe/reloc_driver/run_picker_reloc.py` (driven by the relocate stage with `--skip-build`; the HypoDD/xcorr engine itself is external in `15.PocketQuake`). `detection_test` is the **frozen pilot archive** of the 2016 4-picker comparison — nothing in it is live (`lib/` DEPRECATED; the `reloc_2016_uf*/` dirs hold the pilot's manifests and results).

	`ufpipe` (`src/ufpipe`)	`detection_test`
Role	the self-contained end-to-end catalog + relocation pipeline (incl. `reloc_driver/`)	frozen archive of the 2016 4-picker pilot
Stages	detection → association → augment → PHS → locate → **relocate**	— (nothing live)
Driver	`python -m ufpipe.run_pipeline`	—
Ends at	HypoDD dt.cc (§7); results in `outputs/reloc/`	—

ufpipe's own detection + association cover **KS/KG/GJ/NS** with daily-chunked association (§5), and the `relocate` stage builds its inputs from those outputs (`ufpipe.reloc_inputs`); it no longer consumes `detection_test/lib`. The pipeline is fully self-contained. See §7.

1.2 Two orthogonal "model" dimensions

Dimension	Flag	Values
Picker model	`--model`	`stead`, `original`, `phasenet_plus`, `eqt`, …
Velocity model	`--velmodel`	`kim1983`, `kim2011`

Any picker model can be located with any velocity model. `stead`/`original` are SeisBench PhaseNet weights; `phasenet_plus` is EQNet PhaseNet+ (needs a local EQNet clone).

2 Install & environment

The pipeline runs in a **two-env split** on this server (there is no single "ulsan" env): detection (PhaseNet+/SeisBench, torch) in `eqnet` (Py 3.9); association (PyOcto) + orchestration in `base` (Py 3.12); the relocate stage's GPU xcorr shells out to `pq-gpu` internally.

One-time setup

1. `conda run -n eqnet pip install -e . --no-deps` (detection env)
2. `conda run -n base pip install -e . --no-deps` (association/orchestration env)
3. **HYPOINVERSE**: the external `hyp1.40` binary must be on `PATH` (not pip-installable).
4. **PhaseNet+ only**: a local EQNet clone at `config.EQNET_DIR`.
5. **Fresh machine**: `conda env create -f environment.yml` (env `uf-catalog`), then install the torch build matching your CUDA (`pip install torch --index-url https://download.pytorch.org/whl/cu128`).

After `pip install -e .`, `import ufpipe.config` and `python -m ufpipe.run_pipeline` work from any directory — no `sys.path` hacks.

3 Stages 1–5: detection through location

`STAGES = [detection, association, augment, phs, locate, relocate]` — run in order. Stages 1–5 (below) are functions in `src/ufpipe/core.py`; the relocate stage (6) is `src/ufpipe/relocate.py` and has its own §7. `run_pipeline.py` chains them per year.

#	Stage	In → Out	What it does
1	**detection**	continuous mSEED → `picks/` (per station-day)	AI picker on each station-day across **KS/KG/GJ/NS** (per-year multi-network table, `stations.py`); anti-alias >100 Hz, unify mixed rates, resample to 100 Hz, pick P/S above threshold. GPU. Idempotent (skips done days).
2	**association**	picks → `pyocto_*.csv` + assignment	PyOcto octree associator groups picks into events (gate below), **daily-chunked** (±150 s/day, in-day origins) so it scales to the dense NS array. Coords from the multi-network table.
3	**augment**	assignment → augmented assignment	scan daily picks for orphans PyOcto missed, add to each event so HYPOINVERSE sees the full pick set. Backs up + overwrites in place; idempotent.
4	**phs**	assignment → `UF{year}.phs`	write the HYPOINVERSE phase file from the (augmented) assignment.
5	**locate**	`.phs` → `UF{year}.{sum,arc}`	run HYPOINVERSE (`hyp1.40`) with the chosen velocity model → located catalog + errors.

Pipeline order matters

`augment` runs *between* `association` and `phs`, so HYPOINVERSE sees the full augmented pick set and computes correct `GAP`/`DMIN`/`ERZ`/`RMS`. Do not reorder.

3.1 Source of truth for pick↔event assignment

The **PyOcto assignment (after augmentation)** is authoritative for which `(station, phase)` tuples belong to each event. Do NOT use a legacy time-window pick dump — when two events fall <60 s apart at the same station the earliest pick "wins" and leaks into the wrong event. Downstream (SAC-header writers, magnitude/QC notebooks) must read `pyocto_assignment_{velmodel}_{year}.csv`.

4 Parameters (`src/ufpipe/config.py` — the single disclosed source)

Change defaults here. Everything is one file.

4.1 Detection / DSP

Parameter	Value	Meaning
`SAMPLING_RATE`	100.0 Hz	the pickers' trained rate; all data resampled to it
`ANTIALIAS`	lowpass 45 Hz, 8 corners, zerophase	applied BEFORE down-sampling (>100 Hz KG/NS)
`BANDPASS`	1–40 Hz, 4 corners, causal	legacy detection path; PhaseNet+ runs raw
`P_THRESHOLD`, `S_THRESHOLD`	0.2	SeisBench pick probability threshold
`PNPLUS_MIN_PROB`	0.3	EQNet PhaseNet+ pick threshold
`PNPLUS_NT`	1024×36	PhaseNet+ patch length (whole-day, `cut_patch`)
`MAX_CORES`	24	CPU cap (respects `taskset` affinity)

4.2 Association (PyOcto) — daily-chunked, `ASSOC_GATE` (lenient) vs `ASSOC_GATE_STRICT`

Association runs **one calendar day at a time**: a ±`ASSOC_OVERLAP_S` (150 s) window per day is associated and only events whose *origin* falls in-day are kept (dedup). This is physically equivalent to a single pass (local events are seconds long) but keeps each PyOcto solve to seconds — a whole-year single pass is intractable on the dense ∼200-station NS array (≫1 h, 12 GB). Station coordinates come from the multi-network year table (`stations.py`), so KS/KG/GJ/NS all associate.

NS station epochs — moved stations are SPLIT and association is the rename boundary

22 of the 220 GHBSN (NS) stations were physically relocated during operation (12 m–5.2 km; N108 and N166 twice). Each position epoch is its OWN station: lowercase suffixes from the *first* epoch (`N010a`, `N010b`; a plain code always means never-moved). The canonical epoch table (`ufpipe.ns_epochs`, generated from the user-validated SOTA list `stations/ns/20240715/GHBSN_info_ver202312_modified.csv`) is the single source; the old list carried a 13 km paste error at N019 and the epochs were previously AVERAGED into fictitious midpoints. Detection and the raw picks always use BASE codes (position-free; the archive stores base-code dirs); at the association input, each NS pick is stamped with the epoch code in force at its pick time, and every downstream product — assignment, `.phs`, `.sta`, `station.dat`, SAC filenames and headers — knows only epoch codes. xcorr therefore never pairs waveforms across a move (correct: the differential geometry is broken by it), and responses alias epoch → base code (a move is not a re-equip). The NS_100hz mirror is retired (it was a partial 2021 test build, 8% coverage, and its per-station swap silently dropped stations in other years); NS is read native 200 Hz with the same anti-alias path as GJ.

Parameter (`config`)	lenient	`_STRICT`	Meaning
`REGION_CENTER` ± pad	(35.856,129.224) ± (1.0,1.2)		search area (deg)
`ASSOC_ZLIM`	(0, 30)		depth range (km)
`ASSOC_TIME_BEFORE`	300		origin-search window (s)
`ASSOC_OVERLAP_S`	150		daily-chunk overlap (s)
`n_picks`	4	6	min total picks
`n_p` / `n_s`	2 / 2	3 / 3	min P / S picks
`n_ps`	1	2	min stations with both P+S (depth)
`ASSOC_PICK_MATCH_TOL`	1.5		residual cap (s)

Every value above is **overridable per run** (defaults = config) with matching flags on `association.py` and `run_pipeline.py`: `--gate n_picks=…,n_p=…,n_s=…,n_ps=…`, `--pick-match-tol`, `--overlap-s`, `--zlim Z0,Z1`, `--center LAT,LON`, `--lat-pad`, `--lon-pad`, `--time-before`; `--strict` selects `ASSOC_GATE_STRICT` (ignored if `--gate` is given). The cockpit notebook sets the same knobs via `run_association_year(overrides=…)`. Velocity = `ASSOC_VELMODEL` (`kim2011` 1-D). **Station-code alias**: a picked code missing from the year table but present as `<code>2` (e.g. `BUS`→`BUS2` — same station, renamed code) is remapped so its picks are used, not dropped. Output schema (events `idx,time,x,y,z,picks,latitude,longitude,depth`; assignment `event_idx,pick_idx,residual,station,phase,time`) is unchanged, so `phs`/`locate`/`relocate` consume it as before.

HYPOINVERSE pick weights (stage 4 input). `make_phs` sets the HYPO71 weight code from the picker's own probability (`config.PHS_WEIGHT_SCHEME="probability"`). Design principle (revised 2026-07): *association is the gatekeeper; the weight is only a prior* — every associated pick gets a NONZERO weight, and genuinely bad picks are killed by HYPOINVERSE's own distance + residual re-weighting, which sees the actual misfit. `PHS_WEIGHT_BINS` = even quartiles of the emitted range $[0.2, 1.0]$: $\geq$0.80→0, $\geq$0.60→1, $\geq$0.40→2, else→3 (0 = full weight; **code 4 is never assigned**). S picks one code worse (`PHS_WEIGHT_S_PENALTY`), **capped at 3**. Probabilities are matched to picks by nearest time within `PHS_PROB_MATCH_TOL_S` (50 ms), alias-aware (BUS/BUS2); a pick with no match gets the conservative `PHS_WEIGHT_UNMATCHED_CODE`=3, not full weight. Set `PHS_WEIGHT_SCHEME="phase"` for the legacy fixed P=0/S=1.

The previous scheme (≥0.90/0.70/0.50/0.30, else 4, S+1 uncapped) discarded **17.5%** of associated picks (26% of S) and matched only 74% of probabilities, defaulting the rest to FULL weight; on 2016 it also produced the 92 all-weight-4-P events that segfaulted `hyp1.40`. The revision took 2016 from 11,807 located / 7,830 QC-pass to **13,123 located (99.9% of associated) / 9,440 QC-pass**, with median RMS unchanged (0.05 s).

4.3 Velocity model & location

`DEFAULT_VELMODEL = kim2011`; `kim1983` also available (`data/metadata/velocity/kim1983.csv`, a layered V_p/V_s model; HYPOINVERSE `*.crh` tables live in `data/hypoinv/{kim1983,kim2011}/`). `PHASE_CHANNELS = {P: HHZ, S: HHN}`.

The HYPOINVERSE station file is GENERATED, not hand-maintained. Before locating, `stations.write_hypoinverse_sta(year)` derives `STA/UF<year>_hyp.sta` (fixed-format) and `STA/UF<year>.sta` (CSV, read by `uf_cluster.load_stations`) from the multi-network year table — the single source of truth. `--no-regen-sta` keeps an existing file; the previous hand file is preserved once as `*.legacy`.

Why this matters — a silent and destructive failure mode

A stale or incomplete station list does *not* raise an error. HYPOINVERSE prints `SKIP PHASE CARD WITH UNKNOWN STATION` for every pick at a station it does not know, then drops any event left with < 4 usable phases (`CANT SOLVE`) — and those losses propagate into the `.arc` → ph2dt → `event.dat`/`dt.ct` that HypoDD relocates on. The shipped legacy list held **10** stations for 2010 while the association used **12**: 21% of picks were discarded. Regenerating took 2010 from **640 located / 161 QC-pass** to **791 / 303**. The gap was far larger in later years — the legacy list knew 13 of 46 stations in 2016 (no GJ array) and 44 of 243 in 2021 (no NS array).

4.4 Instrument responses — all four networks (`src/ufpipe/responses.py`)

`responses.py` is to responses what `stations.py` is to coordinates: the single source of truth. `load_inventory(networks=None)` returns one merged `obspy.Inventory` spanning KS/KG/GJ/NS, built master-first so an authoritative entry is never shadowed by a per-station fill. Sources, all under `data/metadata/responses/`:

Network	Directory	Format	Coverage
KS + KG	`master/`	StationXML	`KS_KG_metadata_1.0.2.xml`
KS (7 gaps)	`fetched/extracted/`	SEED RESP	fetched from NECIS
GJ	`gj/`	SEED RESP	30 stations × 3 (MARA)
NS	`ns/`	SEED RESP	200 stations × 3 (MARA)
NS N201–N220	`ns_derived/`	SEED RESP	20 stations × 3, **derived**

Coverage is **100% of the station table for every year 2010–2024**, all three channels. Check it with `python -m ufpipe.responses --coverage <year>`. The ML side reaches the same inventory through `ml_pipeline.load_full_inventory()`.

A missing response is silent rather than loud

Response removal is what turns counts into Wood-Anderson millimetres. A station absent from the inventory raises no error — `remove_response` simply skips it, so the event's ML is the median over whatever stations happened to have metadata. Before GJ and NS responses existed, an ML computed on a 2016 or 2021 catalogue would have quietly used only the KS/KG minority (7 of 46 stations in 2016; 43 of 243 in 2021) while appearing to succeed. Run the coverage check before trusting any ML catalogue.

Renamed stations — why the response lookup is epoch-aware

KMA renames a station when it is re-installed: `BUS` → `BUS2` → `BUS3`, `DAG` → `DAG2`, `USN` → `USN2` — *identical coordinates*, strictly sequential response epochs. `responses.epoch_families()` discovers these from the StationXML coordinates (never from the trailing digit, so `GKP1` and `N001` are safe) and requires the epochs to be **non-overlapping**, which correctly excludes co-located *contemporaries* such as `CHS`/`CSO` and `INC`/`INCA` — genuinely different instruments that must never be aliased. 23 families, 49 codes.

This matters because of one archive quirk: `KS_KG/BUS2/` holds **both** epochs' data, and its miniSEED headers read `BUS` until 2015 — long after the Nov-2010 rename. Association rewrites those picks to `BUS2`, which is right for *position* (identical coordinates, so location and HypoDD are unaffected) but wrong for the *response*, whose epochs are strict. Without the epoch-aware lookup, 89% of 2010's BUS picks silently lost their response. An archive-wide header audit found **this is the only station affected** — `DAG2`, `ULJ2`, `USN2`, `GKP1/2`, `BUS3` and all GJ/NS dirs have headers matching their directory.

`ns_derived/` rests on an assumption — N2xx magnitudes are provisional

The MARA dataless (2020-06-03) predates the `N201`–`N220` deployment (first data 2022.237), so those 20 stations have no authoritative response. All 600 authoritative NS RESP files are byte-identical apart from station code and epoch start date — one instrument configuration for the whole array ($754.3 \times 4{\times}10^5 = 3.0172{\times}10^8$ counts/(m/s), exact) — so `write_derived_ns` clones it for the missing block, stamping each with that station's own first-day-on-disk as the start date. The assumption is that the 2022 expansion kept the same configuration; the N2xx stations do record at the same 200 Hz, in 3 components, at comparable count amplitudes, but there is no manufacturer record here to confirm it. Every generated file carries a `DERIVED by ufpipe.responses` header. Delete the directory and re-derive the moment real metadata arrives.

5 How to run

Notebooks — one tidy folder per year

`python notebooks/make_year_notebooks.py --years 2010-2024` builds, for each year, `notebooks/<year>/00.Run_pipeline_<year>.ipynb` (the 6-stage cockpit: run + check + PyGMT maps, every knob in the top parameters cell) and `01.Record_sections_<year>.ipynb` (record sections with associated P/S picks + a dedicated augmented-events figure), with YEAR/MODEL pre-filled.

`python notebooks/make_year_notebooks.py --status` prints the per-year progress table (picks / assoc / located / QC / dt.cc) read from disk — the way to manage a multi-year run. Kernel `base`; the emitted notebooks are gitignored (the builders are tracked).

Standard runs (from the repo root)

`python -m ufpipe.run_pipeline --model original --years 2024`	one year, all 6 stages
`python -m ufpipe.run_pipeline --model original --years 2010-2024`	the whole record
`python -m ufpipe.run_pipeline --model original --years 2015,2016 --velmodel kim1983`	specific years, kim1983
`python -m ufpipe.run_pipeline --model original --years 2024 --stage-from association`	resume (picks already done)
`python -m ufpipe.run_pipeline --model original --years 2024 --days 1-3`	detection on a few days (testing)
`python -m ufpipe.run_pipeline --model original --years 2024 --strict`	strict association gate
`python -m ufpipe.run_pipeline --model original --years 2016 --stage-from relocate --through dtcc`	relocation only (dt.cc; needs feeder)

CLI flags: `--model` · `--years` (`'2010-2024'` or `'2015,2016'`) · `--velmodel` · `--days` · `--device {cuda,cpu}` · `--workers` · `--strict` · `--force` (allow writing to `stead`) · `--stage-from {detection,association,augment,phs,locate,relocate}` (start mid-chain) · `--gate`/`--pick-match-tol`/… (association, §4.2) · `--qc`/`--xcorr` (relocation, §7) · `--no-regen-sta` (keep an existing station file).

Shared 64-core server — always pin cores

Detection sizes its preprocessing pool and `torch` threads from the process CPU *affinity*, capped by `MAX_CORES=24`. Launch under `taskset` so the whole job stays within budget:

```
OMP_NUM_THREADS=1 taskset -c 0-7 python -m ufpipe.run_pipeline --model original --years 2010-2024
```

Without pinning, PhaseNet+ grabbed ∼49 cores / 193 threads and starved everything. Inference is **GPU-preferred** (warns loudly, never silently falls back to CPU).

Detection is **idempotent** — it skips station-days whose picks already exist, so runs resume safely after an interruption.

5.1 Running each stage separately

Every `ufpipe` stage is also a standalone CLI (`--model --year`), so you can run detection alone, then association alone, etc. Order is `detection` → `association` → `augment` → `phs` → `locate`. Each reads the previous stage's output from disk.

Stage	standalone command
Detection	`python -m ufpipe.detection --model original --year 2024 [--networks KS,KG,GJ,NS] [--days 1-5] [--workers 8]`
Association	`python -m ufpipe.association --model original --year 2024 [--networks KS,KG,GJ,NS] [--workers 8] [--strict]`
Augment	(runs inside `run_pipeline`; or call `core.run_augment_year` from Python)
PHS	`python -m ufpipe.make_phs --model original --year 2024`
Location	`python -m ufpipe.run_hypoinverse --model original --year 2024 --velmodel kim2011`

Or drive the same stage functions from Python / a notebook:

```
from ufpipe import core, config
core.run_detection_year("original", 2024, days=range(1,6))
events, assign = core.run_association_year("original", 2024)
core.write_phs("original", 2024); core.run_hypoinverse_year("original", 2024,
velmodel="kim2011")
```

5.2 Relocation — running the last stage (stage 6)

The relocate stage is **self-fed**: it needs only ufpipe's own per-year detection + association (`pyocto_kim2011_<year>.csv` + assignment). It builds the reloc input store (event-idx SAC + station table) from those via `ufpipe.reloc_inputs`, then hands off to the driver `run_picker_reloc.py --skip-build` (scaffold → HYPOINVERSE → QC → rereference → GPU xcorr → HypoDD dt.cc). If a year's association is missing, the stage preflights and prints exactly what to run.

Step	command
Detection	`conda run -n eqnet python -m ufpipe.detection --model original --year 2016`
Association	`conda run -n base python -m ufpipe.association --model original --year 2016`
Reloc (to QC)	`python -m ufpipe.run_pipeline --model original --years 2016 --stage-from relocate --through hypoinverse` (fast)
Reloc (full)	`python -m ufpipe.run_pipeline --model original --years 2016 --stage-from relocate --through dtcc` (+ xcorr→HypoDD dt.cc; ∼6 h dense)

`--through hypoinverse` stops at the QC'd absolute-location subset; `--through dtcc` continues to the double-difference relocation. `--clean-cache` deletes the ∼tens-of-GB interp cache after dt.cc (essential for multi-year runs). Two override flags expose the stage-6 knobs per run (defaults = the validated values): `--qc "erh=5,erz=5,gap=270,num=5,rms=1.0"` (the `uf_cluster.QC` gate) and `--xcorr`

`"interp_hz=1000,fmin=5,fmax=20,cc_threshold=0.7,pre=0.5,post=0.5,margin=0.5"` (the dt.cc cross-correlation). The HypoDD adaptive-damping/ISTART=2 scheme and the full-run-HYPOINVERSE injection are fixed invariants (not flag-settable). The cockpit notebook (`notebooks/00.Run_yearly_pipeline.ipynb`) sets all of these from its parameters cell. `detection_test/lib/` is DEPRECATED (see `lib/DEPRECATED.md`); the relocation internals are documented in `src/ufpipe/reloc_driver/PIPELINE.md` + `study_guide.pdf`.

6 Outputs & layout

All pipeline products go under `outputs/models/<model>/` (regenerable; gitignored):

Path	Contents
`.../detection_location/<year>/picks/`	per-station-day picks (stage 1)
`.../pyocto/pyocto_{vm}_{year}.csv`	associated events (stage 2)
`.../pyocto/pyocto_assignment_{vm}_{year}.csv`	pick→event assignment (authoritative)
`.../station_table/stations_{year}.csv`	per-year station roster (discovered from metadata)
`.../HypoInv/PHS/UF{year}.phs`	HYPOINVERSE phase file (stage 4)
`.../HypoInv/{vm}/UF{year}.{sum,arc,prt}`	located catalog + errors (stage 5)
`outputs/reloc/reloc_{year}_uf[_{model}]/results/`	dt.cc-relocated catalog + links (stage 6)

`{vm}` = `config.PYOCTO_VELMODEL` = **kim2011** for current daily-chunked runs; legacy pre-2026-07 whole-year artifacts on disk are named `kim1983`.

Inputs (metadata by kind, 2026-07): waveforms in `KS_KG/ GJ/ NS/ NS_100hz/` (station dirs at each root); station tables in `data/metadata/stations/{ks_kg,gj,ns,kigam}/` (+ the per-year derived cache `stations/derived/`); instrument responses for all four networks in `data/metadata/responses/{master,fetched,gj,ns,ns_derived}/` (§4.4); velocity model in `data/metadata/velocity/`; HYPOINVERSE control inputs (STA/`kim*`) in `data/hypoinv/`.

7 Stage 6: relocation — HypoDD dt.ct + dt.cc

The 6th ufpipe stage: turn the QC'd absolute-location catalog into a **double-difference relocated** catalog for sub-100 m relative precision.

```
python -m ufpipe.run_pipeline --model original --years 2016 --stage-from relocate
--through dtcc --clean-cache
```

`ufpipe.relocate` (`src/ufpipe/relocate.py`) is *self-fed*: it preflights ufpipe's OWN per-year association, builds the reloc inputs from it (`ufpipe.reloc_inputs` — SAC store, pyocto tables, station table, merged archive), then invokes the validated orchestrator `src/ufpipe/reloc_driver/run_picker_reloc.py --skip-build`, which drives the external HypoDD/xcorr engine. Internal stages (all KS/KG/GJ/NS):

One id end-to-end — the manifest era (2026-07-29)

Every event is keyed by `id = 200000 + event_idx` (ufpipe's global PyOcto index) in EVERY engine product: `.sum`, `.arc`, `event.dat`, `dt.ct`, `dt.cc`, `hypoDD.reloc`. The staging step writes `event_manifest.csv` into the engine run dir and the engine's `evmap` assigns cuspids from it — never from position over sorted directories, which silently shifted ids whenever stale staging or a same-second doublet changed the directory count (the 2010/2011 failures). The QC injection is therefore a pure id *subset* (byte-identical row copy, no renumbering, no time matching), and `hypoDD.reloc.dtcc` joins directly to the PyOcto tables and `event_sac/<event_idx>/` with no lookup. All hypoDD solves everywhere use **LSQR** (`ISOLV=2`): the recompiled large-array binary makes SVD pathologically slow at any size.

#	Stage	What it does
1	SAC store	event-idx-keyed native-rate SAC waveforms from the associated picks (`event_sac/`)
2	HYPOINVERSE	absolute location of all UF-box events (kim2011) → `.sum`
3	QC	`uf_cluster.QC`: keep `erh<5, erz<5, gap<270, num>5, rms<1.0`
4	inject full `.sum`/`.arc`/STA	QC subset reuses the full-run HYPOINVERSE solution + station file (no redundant re-run)
5	ph2dt / dtct	build `event.dat` + catalog differential times `dt.ct` from the injected `.arc`
6	re-reference	stamp each SAC's origin from the (full-run) HYPOINVERSE `.sum`
7	xcorr (GPU)	cross-correlation differential times `dt.cc` (band 5–20 Hz, interp 1000 Hz, cc≥0.7)
8	HypoDD	kim2011, ISTART=2, ISOLV=2 (LSQR), adaptive damping (CND 60–80), HypoDD v2.1beta (MAXDATA=15M) → `hypoDD.reloc`

Relocation outputs (symlinked into the working dir)

Each run publishes `outputs/reloc/reloc_<year>_uf[_<p>]/results/` with symlinks to the external run tree: `hypoDD.reloc.dtcc` (dt.ct+dt.cc — **the deliverable**), `HypoInv.full.sum`, `HypoInv.qc.sum`, …"dt.cc-resolved" = events keeping ≥1 surviving cross-correlation link — the highest-precision subset. `hypoDD.reloc.dtct` (dt.ct-only) is an *optional diagnostic baseline*: it re-runs HypoDD on catalog differential times only (heavier, no cc) and is non-fatal + time-bounded, so it may be absent when slow — the dt.cc result is unaffected.

The invariant that matters (see PIPELINE.md)

Origins flow into dt.cc: `rereference` stamps SAC origins from a `.sum`, and xcorr stores $dt = (t_1 + \text{shift} - ot_1) - (t_2 - ot_2)$ — subtracting those origins. Use the *full-run* HYPOINVERSE solution QC gated on (never a redundant re-run), or the dt.cc *values* are corrupted. This is enforced by `run_picker_reloc.py::inject_full_hypoinverse`.

8 Robustness & invariants

- **Network growth is automatic.** Stations are *discovered from StationXML + on-disk coverage* each month; the deployed network for that period is what enters the run. No hard-coded station lists.
- **Mixed sampling rates handled.** >100 Hz stations (KG/NS at 200 Hz) are anti-alias-lowpassed then resampled to the pickers' 100 Hz. The anti-alias filter is applied BEFORE down-sampling (never after).
- **Velocity model, thresholds, and gates are epoch-invariant by choice** ⇒ the catalog stays internally comparable across years.
- **Idempotent & resumable** at every stage; `--stage-from` restarts mid-chain.
- **Do not edit the `stead` reference run** (it is the baseline). Scripts refuse `--model stead` writes unless `--force`.

9 Troubleshooting / provenance checks

Symptom	Check
Detection finds no waveforms	`python -c "import ufpipe.stations as s; print(s.build_year_table(YEAR).net.value_counts())"` — the per-year multi-network table must list stations, and each row's `archive` dir must exist.
`ImportError: ufpipe` / `uflib`	`pip install -e . --no-deps` in **both** `eqnet` and `base`; `import ufpipe.config; print(__file__)` must be `src/ufpipe`.
`ModuleNotFoundError: pyocto`	you ran association in `eqnet` — PyOcto lives in `base`. Detection → `eqnet`; association → `base`.
`InternalMSEEDWarning: ...Steim2 failed`	corrupt Steim2 record(s) in the archive (e.g. KG.MKL 2010): the samples still decode; the reader collapses the per-record spam into one summary line per station-day and reads anyway. Informational — the named station-days are the QC signal.
GJ/NS station silently yields no picks	mid-day sampling-rate changes (100/200/1000 Hz) and int32/float64 records refuse to merge — handled since 2026-07 (rate + dtype unified *before* merge in both readers).
`relocate` aborts at preflight	ufpipe's own per-year association is missing for that (model, year) — run `ufpipe.detection` (eqnet) then `ufpipe.association` (base); the preflight prints the exact commands.
Wrong `pipeline` imported	`ufpipe` is renamed precisely to avoid the collision with the external korea-cluster-relocation `pipeline`. Verify `ufpipe.config.__file__` is under `src/`.
Events look over/under-associated	`ASSOC_GATE` vs `--strict` (`ASSOC_GATE_STRICT`); `ASSOC_PICK_MATCH_TOL` is the primary knob.
GPU not used	inference is GPU-preferred and warns loudly; check the torch/CUDA install (never silently CPU).
Sparse early years, few events	expected — KS/KG-only network (2010–2015) has larger azimuthal gaps; GJ joins 2016, the dense NS array 2017+; the QC gate honestly reflects what the network resolves.
`.sum` stops mid-year; months missing	`hyp1.40` **segfaulted** partway (2016 once died on 25 Jun, dropping the whole Gyeongju sequence): it crashes in `hytrl_` on any event whose P picks are ALL weight code 4 (only possible under the probability weight scheme, every P below the last `PHS_WEIGHT_BINS` threshold). Fixed 2026-07: `write_phs` skips such unlocatable events (logged to `PHS/UF<year>.phs.skipped`) and `run_hypoinverse_year` now raises on a nonzero exit code instead of shipping the partial `.sum`. If you see this, regenerate from stage `phs`.

Companion docs: `docs/overview.md`, `docs/pipeline.md`, `docs/how-to-run.md`, `docs/directory-structure.md` · `src/ufpipe/README.md` · relocation pipeline:

`src/ufpipe/reloc_driver/PIPELINE.md` + `study_guide.pdf`.